*****In btween extends Thread and implements Runnable which one is better and Why ?**

Ans : Both are the ways in java.lang package to create a Thread.
In between extends Thread and implements Runnable, implements
Runnable is more better approach due to the following reasons :

a) In extends Thread we cannot extend any other class (MI is not supported by using class) but in implements Runnable we can extend only one class.

b) extends Thread provides tight coupling approach so to create multiple threads we need multiple sub class objects on the other
hand implements Runnable provides loose coupling approach so for creating multiple threads we need only sub class object.

```
Example :
Case 1 :
--------
class Test extends Thread
{
}

Test t1 = new Test();
t1.start();
Test t2 = new Test();
t2.start();

Case 2:
-------
class UserThread implements Runnable
{
}

UserThread ut = new UserThread();

Thread t1 = new Thread(ut);
Thread t2 = new Thread(ut);
```

3) By using extends Thread we can't use Lambda Expression but using implements we can use Lambda Expression

4)If a class extends from Thread class then all the Thread class methdos are available in the sub class but if we use implements Runnable then Thread class methos are not available.

---

volatile Keyword in java : [Reading the value of the variable from the main memory not from the cache memory]

While working in a multithreaded environment multiple threads can perform read and write operation with common variable/data (chances of Data inconsistency so use synchronized OR AtomicInteger) concurrently.

In order to store the value temporarly, Every thread is having local cache memory (PC Register) but if we declare a variable with volatile modifier then variable's value is not stored in a thread's local cache; it is always read from the main memory.

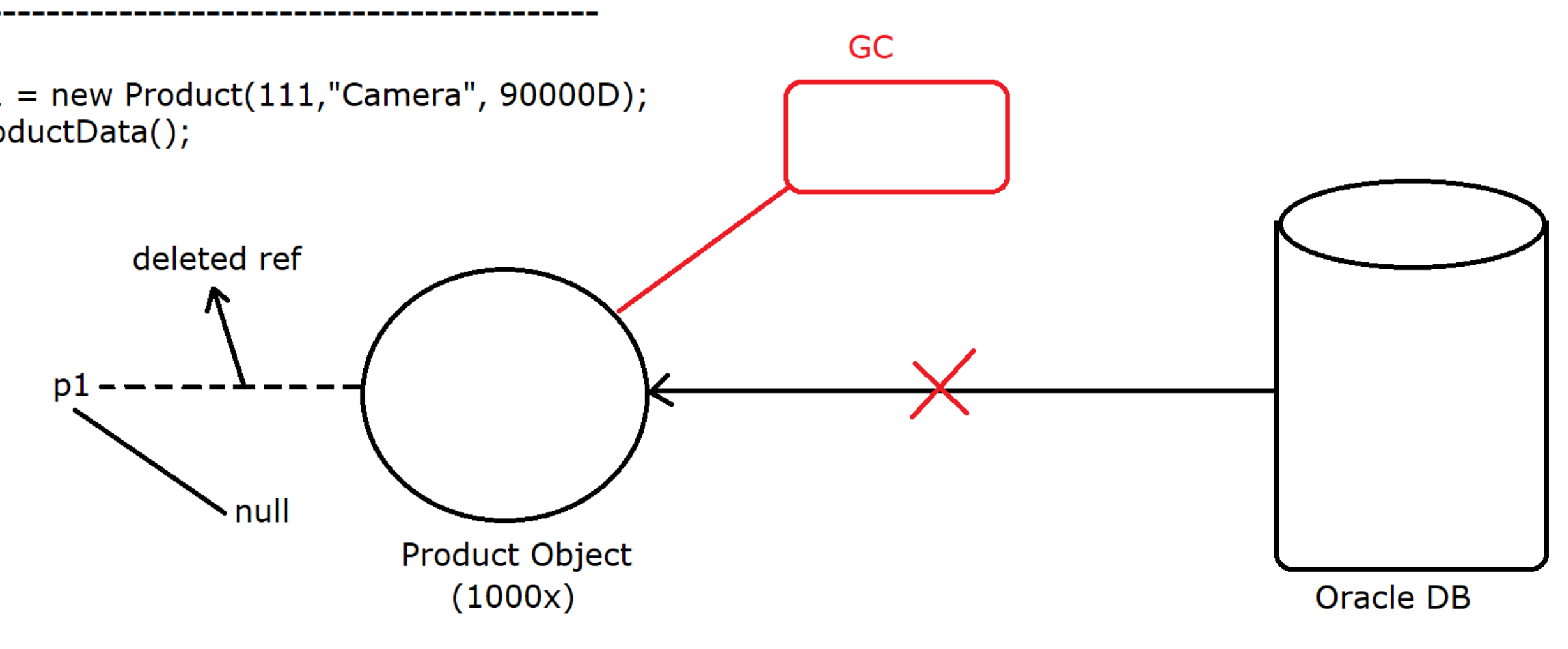
So the conclusion is, a volatile variable value is always read from and written directly to the main memory, which ensures that changes made by one thread are visible to all other threads immediately.

```
//Program :
-------------
package com.ravi.interrupt;

class SharedData
{
    private volatile boolean flag = false;

    public void startThread()
    {
        //Writer thread
        Thread writer = new Thread(() ->
        {
            try
            {
                IO.println("Writer thread started");
                Thread.sleep(5000);  //Writer thread will go for 5 sec waiting state
                this.flag = true;
                IO.println("Writer thread make the flag value as true");
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        });


        //reader thread
        Thread reader = new Thread(() ->
        {
            IO.println("Reader thread started!!!");
            while (!flag) //Cache memory still the value of flag is false, if not declared with
volatile
            {

            }
            IO.println("Reader thread got the updated value");
        });

        writer.start();
        reader.start();
    }

}

public class VolatileExample
{
    public static void main(String[] args)
    {
        new SharedData().startThread();
    }
}
```

---

protected void finalize() throws Throwable :

```
Product p1 = new Product(111,"Camera", 90000D);
p1.readProductData();

p1 = null;
```



It is a predefined non static method of Object class.

Garbage Collector automatically call this method just before an object is eligible for garbage collection to perform clean-up activity.

Here clean-up activity means closing the resources associated with that object like file connection, database connection, network connection and so on we can say resource de-allocation.

Note :- a) JVM calls finalize method only one per object.

b) This method is deprecated from java 9V. [try with resouces]

c) Calling finalize() method on a particular object is an indication that Object is eligible for GC

```
package com.ravi.finalize;

record Product(Integer id, String name, Double price)
{
    @Override
    public void finalize()
    {
        IO.println("Resourses are closed, Product Object is eligible for GC");
        //db.close();   file.close();  network.close();
    }
}

public class FinalizeDemo
{
    public static void main(String[] args) throws InterruptedException
    {
        Product p1 = new Product(111, "Camera", 12456D);
        IO.println(p1);

        p1 = null;

        System.gc();

        Thread.sleep(5000);

        IO.println(p1);

    }

}
```

Note : Here Product Object is eligible for GC then automatically finalize() method is invoked.

** What is the difference between final, finally and finalize()

final :

It is used to provide some kind of restriction, by using final modifier we can declare our class, Method and Field/variable as a final.

finally :

If we open any resourse as a part of try block then it should compulsory closed inside finally block, [If We close the resourses inside try block, It is always Risky] for resourse handling purpose.

finalize() :

It is a method of Object class, We should override this method in the corresponding class to perform cleanup activity, GC will automatically call this method just before object destruction.

Collections Framework [40-45 % IQ] :

---

* It is a **data structure of jav**a which is implemnted by using **java.util** package.

* The main purpose of Data structure to reduce the **"Time Complexity"** while performing various operations on data like searching, sorting, insertion, deletion and so on.

* The simple meaning of **"Collections"** is to work OR deal with **Objects.**

* In entire Collections Framework, We can deal with **two types of Objects.**

1) Single OR Individal Object (Collection interface)
2) Multiple OR Group of Objects (Map interface)

* In order to work with Single OR Individal Object we should use a predfined interface called **Collection available in java.util package from JDK 1.2V.**

```
Example :
-----------
Collection coll = new Vector();
  coll.add("ravi@gmail.com");
  coll.add("Java");
  coll.add(12);
  coll.add(null);
  IO.println(coll);
```

* In order to work with multiple OR group of Objects we should use a predfined interface called Map<K,V> available in java.util package from JDK 1.2V.

```
Example :
-----------
Map map = new Hashtable();
  map.put("ravi@gmail.com","ravi1234");
  map.put("raj@gmail.com","raj1234");
  map.put("rahul@gmail.com","rahul1234");
  IO.println(map);
```

Basic of Array : Primitive & **Reference type**

hashCode() & equals()

Comparable & Comparator